CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment7

StudentName:Karan **UID:23BCS11811**
Branch:BECSE **Section/Group:KRG3A**
Semester: 6th **DateofPerformance:09/03/26**
Subject Name: Full Stack Development **Subject Code: 23CSH-309**

Aim:
To understand and implement JPA entity mapping, advanced querying, fetch strategies, and databaseperformanceoptimizationtechniquesinabackendapplicationbybuildingefficient data access layers using Spring Boot, JPA, and Hibernate.

Objective:
1. ImplementJPAEntityMapping
- CreateentityclassesusingJPAannotations.
- EstablishOne-to-Manyrelationshipsbetweenentities.
- UnderstandhowORMmapsobjectstorelationaldatabasetables.

2. PerformAdvanced Queries
- WriteandexecuteJPQLqueries.
- Implementcursor-basedpaginationforefficientdataretrieval.
- Understandtheimportanceoflimitingresultsetsinlargedatasets.

3. ExploreFetchStrategies
- ImplementLazyandEager fetching.
- Analyzehowdifferentfetchstrategiesimpactapplicationperformance.
- IdentifyandresolvetheN+1queryproblem.

4. ImproveBackendPerformance
- Applydatabaseindexingforfrequentlyqueriedfields.
- Optimizequeryexecutionusingquerytuningtechniques.
- Measureandcompareperformanceimprovements.

5. ApplyConceptsinProject (HealthHub)
- IntegrateoptimizedentityrelationshipsandqueriesintotheHealthHubbackendproject.
- Evaluateperformanceimpactinrealapplicationscenarios.

Implementation/Code:PatientController.j ava:

```
packagecom.example.healthhub.controller;

import com.example.healthhub.dto.*;
importcom.example.healthhub.service.*;
importorg.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
importorg.springframework.data.domain.Page;

@RestController
@RequestMapping("/patients")
publicclassPatientController{

   @Autowired
   privatePatientServicepatientService;

   @PostMapping
   publicPatientDTOcreatePatient(@RequestBodyPatientDTOdto){
      return patientService.savePatient(dto);
   }

   @GetMapping
   publicPage<PatientDTO>getPatients(
         @RequestParam int page,
         @RequestParam int size) {
      returnpatientService.getPatients(page,size);
   }

   @GetMapping("/{id}")
   publicPatientDTOgetPatient(@PathVariableLongid){ return
      patientService.getPatient(id);
   }
}
```

PatientRepository.java:

```
packagecom.example.healthhub.repository;

import com.example.healthhub.entity.Patient;
importorg.springframework.data.jpa.repository.*;
importorg.springframework.data.repository.query.Param;
import org.springframework.data.domain.*;
```

```
importjava.util.*;

publicinterfacePatientRepositoryextendsJpaRepository<Patient,Long>{

    @Query("SELECTpFROMPatientpWHEREp.age>:age")
    List<Patient>findPatientsOlderThan(@Param("age")intage);

    @Query("SELECTpFROMPatientpLEFTJOINFETCHp.appointments")
    List<Patient> findAllWithAppointments();

    @Query("SELECTpFROMPatientpLEFTJOINFETCHp.appointmentsWHEREp.id
= :id")
    Optional<Patient>findByIdWithAppointments(@Param("id")Longid);

    Page<Patient> findAll(Pageable pageable);

    Optional<Patient>findByEmail(Stringemail);
}
```

PatientService.java:

```
packagecom.example.healthhub.service;

import com.example.healthhub.entity.*;
import
com.example.healthhub.dto.*;importcom.ex
ample.healthhub.repository.*;
importorg.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
importorg.springframework.data.domain.*;

import java.util.*;

@Service
publicclassPatientService{

    @Autowired
    privatePatientRepositorypatientRepository;

    publicPatientDTOsavePatient(PatientDTOdto){ Patient
        patient = new Patient();
        patient.setName(dto.getName());
        patient.setEmail(dto.getEmail());
```

```
patient.setAge(dto.getAge());
```

```
        Patientsaved=patientRepository.save(patient); return
        convertToDTO(saved);
    }

    publicPage<PatientDTO>getPatients(intpage,intsize){
        Pageable pageable = PageRequest.of(page, size);return
        patientRepository.findAll(pageable)
                .map(this::convertToDTO);
    }

    publicPatientDTOgetPatient(Longid){
        Patientpatient=patientRepository.findByIdWithAppointments(id)
                .orElseThrow(()->newRuntimeException("Patientnotfound")); return
        convertToDTO(patient);
    }

    publicPatientDTOconvertToDTO(Patientpatient){
        PatientDTO dto = new PatientDTO();

        dto.setId(patient.getId());
        dto.setName(patient.getName());
        dto.setEmail(patient.getEmail());
        dto.setAge(patient.getAge());

        if (patient.getAppointments() != null) {
            List<String>dates=patient.getAppointments()
                    .stream()
                    .map(Appointment::getDate)
                    .toList();
            dto.setAppointmentDates(dates);
        }

        returndto;
    }
}
```

Output:

POST http://localhost:8080/patients Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "name": "Sana",
  "email": "sana@gmail.com",
  "age": 35
}
```

Body Cookies Headers (5) Test Results 200 OK 225 ms 244 B

{} JSON Preview Visualize

```
{
    "age": 35,
    "appointmentDates": null,
    "email": "sana@gmail.com",
    "id": 4,
    "name": "Sana"
}
```

GET http://localhost:8080/patients/3 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results 200 OK 660 ms 244 B

{} JSON Preview Visualize

```
{
    "age": 30,
    "appointmentDates": [],
    "email": "david@gmail.com",
    "id": 3,
    "name": "David"
}
```

GET http://localhost:8080/patients?page=0&size=2 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results 200 OK 47 ms 642 B

JSON Preview Visualize


```
{
    "content": [
        {
            "age": 21,
            "appointmentDates": [],
            "email": "tanureet@gmail.com",
            "id": 1,
            "name": "Tanureet"
        },
        {
            "age": 21,
            "appointmentDates": [],
            "email": "ana@gmail.com",
            "id": 2,
            "name": "Ana"
        }
    ],
    "empty": false,
    "first": true,
    "last": false,
    "number": 0,
    "numberOfElements": 2,
    "pageable": {
        "offset": 0,
        "pageNumber": 0,
        "pageSize": 2,
```


GET http://localhost:8080/patients?page=1&size=2 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results 200 OK 19 ms 638 B

JSON Preview Visualize


```
{
    "content": [
        {
            "age": 30,
            "appointmentDates": [],
            "email": "david@gmail.com",
            "id": 3,
            "name": "David"
        },
        {
            "age": 35,
            "appointmentDates": [],
            "email": "sana@gmail.com",
            "id": 4,
            "name": "Sana"
        }
    ],
    "empty": false,
    "first": false,
    "last": true,
    "number": 1,
    "numberOfElements": 2,
    "pageable": {
        "offset": 2,
        "pageNumber": 1,
        "pageSize": 2,
```

LearningOutcome:

1. AbilitytodesignandimplementJPAentityclassesusingappropriateannotationsand relationships, particularly One-to-Many mappings.

2. UnderstandingofObject-RelationalMapping(ORM)conceptsandhowJavaobjects are mapped to relational database tables.

3. AbilitytowriteandexecuteadvancedJPQLqueriesand implementefficientdata retrieval techniques such as cursor-based pagination.

4. Understandingtheimportanceofoptimizingqueriesbylimitingresultsetswhen working with large datasets.

5. AbilitytodifferentiatebetweenLazyandEagerfetchingstrategiesandanalyzetheir impact on application performance.